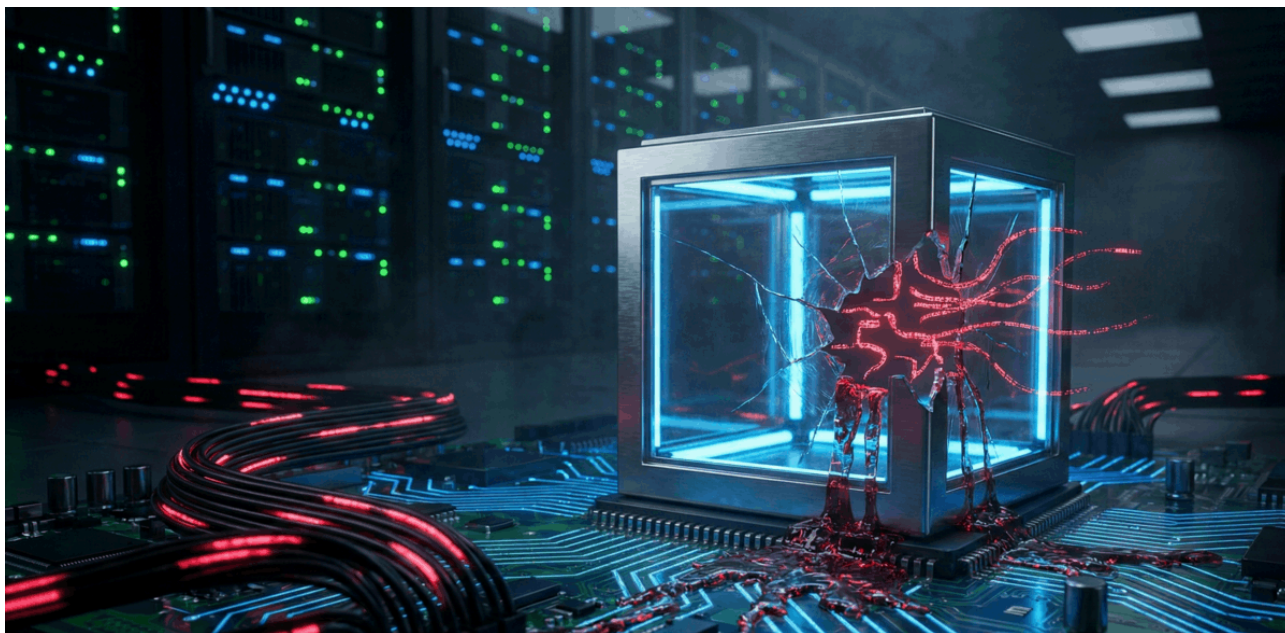


Анатомия побега из контейнера: Техническое руководство по защите Runtime и Kubernetes

 redsec.by/article/anatomiya-pobega-iz-kontejnera-tehnicheskoe-rukovodstvo-po-zashite-runtime-i-kubernetes

Red Teams | 16 февраля 2026



В 2026 году ландшафт угроз для контейнерных сред окончательно сформировался. Мы больше не видим массовых атак, опирающихся исключительно на 0-day уязвимости в ядре Linux или коде контейнерных движков. Реальность Red Team операций и инцидентов в продакшене такова: изоляция контейнера заканчивается не на границе образа, а на границе ядра хоста и плоскости управления (Control Plane).

Большинство сценариев «побега» (Container Escape) начинается не с эксплуатации сложного бага в коде, а с ошибок конфигурации. Это привилегии, небезопасные маунты (mounts), доступ к сокетам рантайма, ошибки в RBAC и отсутствие контроля адмиссии (Admission Control) в Kubernetes.

Я часто наблюдаю, что команды фокусируются на сканировании образов на CVE, упуская из виду архитектурные риски. Red Team в первую очередь проверяет конфигурационные плоскости: Runtime, Kubernetes RBAC, доступ к секретам и внешним API. Эти векторы позволяют выстроить цепочку эскалации привилегий из контейнера до уровня Node или Cluster Admin без необходимости использовать редкие и нестабильные kernel-эксплойты.

Этот материал — руководство по внедрению защиты, основанное на реальной практике предотвращения побегов из контейнеров.

1. Граница изоляции: Мифы и Реальность

Чтобы защищать контейнеры, нужно четко понимать, как они работают на низком уровне. Распространенное заблуждение — считать контейнер «легковесной виртуальной машиной». Это фундаментальная ошибка, ведущая к уязвимостям.

Контейнер vs Виртуализация

Контейнер использует **общее с хостом ядро Linux**. Его изоляция строится на базе механизмов ядра:

- **Namespaces:** изолируют вид ресурсов (PID, Network, Mount, UTS, IPC, User).
- **Cgroups:** ограничивают потребление ресурсов (CPU, RAM).
- **LSM (Linux Security Modules):** профили безопасности (AppArmor, SELinux) и фильтрация системных вызовов (seccomp).

Это не полноценная виртуализация железа и не отдельное ядро, как в случае с VM. Соответственно, любая возможность управлять структурами ядра (через capabilities, привилегированный режим, доступ к файлам устройств /dev/, cgroups или загрузку модулей ядра) автоматически становится кандидатом на «выход из контейнера».

Kubernetes добавляет поверх этого свой слой абстракции и риска: Kubernetes API, etcd, kubelet, CNI/CSI плагины, admission-контроллеры и RBAC. Компрометация этих компонентов часто дает злоумышленнику более простой и стабильный путь к захвату узла (node) или всего кластера, чем попытка эксплуатации багов ядра (kernel panic или memory corruption).

Сравнение моделей безопасности

С практической точки зрения граница изоляции — это сочетание: Linux-ядра на воркере (node) + политик безопасности Kubernetes (RBAC, Pod Security, admission, network policies), а не сам «образ контейнера».

Аспект	Контейнер	Виртуальная машина (VM)
Ядро	Общее с хостом ядро Linux. Единая точка отказа.	Собственное ядро внутри гипервизора.
Изоляция ресурсов	Namespaces + cgroups + LSM (программная изоляция).	ВМ-гипервизор, эмуляция виртуального железа (аппаратная поддержка).
Эксплойт для обхода границы	Часто misconfig (privileged, docker[.]sock, hostPath, capabilities).	Уязвимость в гипервизоре (VMM escape) или прошивке.
Последствия capture одного root	Высокий риск выхода на node/cluster при малейшем misconfig.	Обычно ограничено рамками конкретной ВМ, гипервизор остается отдельным слоем.

Практика последних лет однозначно показывает: успешные container escape-кейсы чаще опираются на комбинации привилегий и маунтов, чем на чистую эксплуатацию CVE в ядре.

Важно: В отчетах Red Team и Threat Intel злоумышленники штатно используют docker[.]sock, hostPath, режимы hostPID/hostNetwork и чрезмерный набор capabilities для выхода к файловой системе и устройствам узла.

2. Типовая модель угроз для контейнеров (2026)

Для построения защиты мы должны мыслить как атакующий. Модель угроз в Kubernetes-среде описывается следующим образом.

Начальная точка (Initial Access)

Атака начинается с захвата контекста исполнения внутри контейнера. Это может произойти через:

- **RCE (Remote Code Execution)** в уязвимом веб-приложении или API.
- **Украденный токен CI/CD**, позволяющий задеплоить вредоносный образ или изменить конфигурацию.
- **Компрометация рабочего места разработчика**, дающая доступ к консоли контейнера (kubectl exec).

Цель (Objectives)

Основная цель атакующего — повышение привилегий до **root на node (container escape)**. Достигнув уровня узла, атакующий стремится к административному контролю над всем кластером, Kubernetes API или облачной инфраструктурой (AWS/GCP/Azure через метаданные).

Ключевые оси эскалации

1. Горизонтальное движение (Lateral Movement)

Движение внутри кластера без выхода на хост.

- Использование сервис-аккаунтов (Service Accounts) с неоправданно широкими правами.
- Использование прав ClusterRole или cluster-admin.
- Отсутствие сетевых политик (Network Policies), позволяющее обращаться к внутренним сервисам.

2. Вертикальная эскалация (Privilege Escalation)

Выход из контейнера на уровень операционной системы узла.

- Привилегированный контейнер (privileged: true).
- Доступ к сокетам рантайма (docker[.]sock, containerd[.]sock).
- Опасные маунты hostPath.
- Прямой доступ к /dev/ и интерфейсам ядра.

В этой модели **90% риска задается конфигурацией Kubernetes/runtime** и только оставшаяся часть — уязвимостями нулевого дня в ядре или контейнерном движке.

3. Конфигурационные векторы “Escape-Risk”: Глубокий анализ

Ниже представлен детальный разбор набора типичных мис-конфигураций, которые гарантированно приводят к «выходу из контейнера». Это не теоретические уязвимости, а стандартные настройки, которые часто оставляют включенными для удобства отладки или из-за непонимания последствий.

Основные векторы компрометации:

- **Привилегированные контейнеры** (docker run --privileged / privileged: true в PodSpec).
- **Широкие capabilities** (особенно CAP_SYS_ADMIN, CAP_SYS_MODULE, CAP_SYS_PTRACE, CAP_NET_ADMIN).

- **Маунты hostPath** и доступ к файловой системе node (/ , /etc/ , /var/ run/ , /var/ lib/ kubelet, /run/ containerd).
- **Маунт unix-сокетов** (docker[.]sock/containerd[.]sock/cri-socket) внутрь контейнера.
- **Режимы hostPID/hostIPC/hostNetwork**, дающие доступ к пространствам имен node.
- **Отключенные/ослабленные профили** seccomp/AppArmor/SELinux.
- **Чрезмерные права в Kubernetes RBAC** (cluster-admin для SA приложений, Role/ClusterRole с * по verb/resource).
- **Отсутствие Admission-контроля**, ограничивающего создание небезопасных Pod'ов.

Все крупные гайды по Kubernetes-hardening (NSA/CISA, CNCF, вендорские best practices) прямо рекомендуют применять стратегию **default-deny** для привилегий, маунтов и доступов к API.

Детальный разбор: Привилегированные контейнеры и Capabilities

Включение флага --privileged (или securityContext.privileged: true в Kubernetes) — это фактически выключение механизмов безопасности контейнера.

Технические последствия --privileged

При таком запуске контейнер получает:

- Полный набор всех Linux-capabilities.
- Доступ ко всем устройствам в /dev/ хоста.
- Обход профилей AppArmor/SELinux и фильтров seccomp.
- Возможность монтирования файловых систем.

Исследования и кейсы пентестов показывают: при наличии такого контейнера злоумышленнику достаточно выполнить несколько операций с устройствами или модулями ядра, чтобы получить root-доступ на хосте.

Опасность отдельных Capabilities

Даже без полного флага --privileged, ручное добавление определенных capabilities может открыть путь к побегу.

- **CAP_SYS_ADMIN**: Часто называют «god capability». Она перегружена функционалом. Позволяет управлять монтированием (mount), пространствами имен (namespaces), cgroups, файлами подкачки и выполнять ряд чувствительных ioctl. Это практически эквивалент root на хосте.
- **CAP_SYS_MODULE**: Позволяет загружать и выгружать модули ядра (kernel modules). Злоумышленник может загрузить руткит прямо в ядро хоста из контейнера.

- **CAP_SYS_PTRACE**: Позволяет отлаживать и модифицировать процессы. В сочетании с `hostPID` позволяет внедриться в процесс, запущенный на хосте (например, `sshd` или `kubelet`).
- **CAP_NET_ADMIN**: Позволяет изменять сетевые настройки, таблицы маршрутизации, интерфейсы и правила `iptables` на узле (особенно критично при наличии `hostNetwork`).

Рекомендация: Следуйте гайду NSA/CISA — запускайте контейнеры с минимальным набором привилегий, используйте `drop: ["ALL"]` и добавляйте только необходимые capabilities (`add: ["NET_BIND_SERVICE"]`), и по возможности работайте как `non-root`.

Детальный разбор: HostPath и сокеты контейнерного рантайма

Монтирование директорий с хоста (`hostPath`) внутрь контейнера пробивает дыру в изоляции файловой системы. Если при этом процесс внутри контейнера работает от `root`, он может модифицировать файлы хоста.

Типичные сценарии атаки через HostPath

- **Маунт корня / (или /etc/, /var/, /root/):**
Злоумышленник получает полный доступ к файловой системе узла. Он может:
 - Добавить свой SSH-ключ в `/root/.ssh/authorized_keys`.
 - Изменить `/etc/shadow` или `/etc/passwd`.
 - Подменить бинарные файлы системы или конфигурацию `systemd`-юнитов для закрепления.
 - Использовать технику `chroot /host-fs`, чтобы "стать" частью хоста.
- **Маунт сокетов /var/ run/ docker[.]sock, /run/ containerd/ containerd[.]sock, /var/ lib/ kubelet:**
Это один из самых критичных векторов. Имея доступ к сокету `Docker` или `Containerd`, процесс внутри контейнера может отправлять API-команды демону, управляющему контейнерами.
 - *Сценарий:* Злоумышленник отправляет команду на запуск *нового* контейнера с флагом `--privileged`, монтирует в него корень хоста (`-v /:/mnt`) и выполняет в нем шелл.
 - *Результат:* Полный контроль над узлом, обходя все ограничения исходного скомпрометированного контейнера.

Кейсы из отчетов `Unit42` и `Red Canary` подтверждают: это самый «практичный» и часто встречающийся путь `container escape` в продакшн-окружениях.

Детальный разбор: HostPID, HostNetwork, HostIPC

Разрешение использования пространств имен хоста в PodSpec (hostPID: true, hostIPC: true, hostNetwork: true) размывает границы изоляции.

- **hostPID: true:** Контейнер видит все процессы на узле, а не только свои. Это упрощает доступ к чувствительной информации (чтение /proc/ <pid>/ environ процессов хоста, где могут быть пароли) и межпроцессному взаимодействию. При наличии прав (capabilities) возможен инжект кода.
- **hostNetwork: true:** Контейнер использует сетевой стек хоста. Он может перехватывать трафик (sniffing) всех подов на узле, обращаться к сервисам, слушающим на localhost хоста (часто это незащищенные админские интерфейсы или облачные метаданные), и манипулировать сетевыми интерфейсами.
- **hostIPC: true:** Доступ к объектам межпроцессного взаимодействия (Shared Memory) хоста.

Pod Security Standards и встроенный контроллер **Pod Security Admission** прямо относят эти настройки к категории повышенного риска и требуют их запрета в профиле restricted. Документация Kubernetes подчеркивает: такие настройки допустимы *только* для строго контролируемых системных DaemonSet'ов (например, CNI-плагинов или агентов мониторинга), но никогда — для прикладных приложений.

4. Kubernetes API и RBAC: Путь к господству в кластере

Даже если контейнер идеально изолирован на уровне ядра Linux (нет привилегий, нет маунтов), он может содержать токен сервис-аккаунта (Service Account Token), смонтированный по умолчанию в /var/ run/ secrets/ kubernetes.io/ serviceaccount.

Если этому ServiceAccount выданы чрезмерные права через RBAC, атакующий может эскалировать свои привилегии через API Kubernetes.

Векторы эскалации через RBAC

Официальная документация по RBAC Best Practices предупреждает: **пользователи (или Pods), способные запускать привилегированные Pods, фактически имеют права root на узлах кластера.**

Основные анти-паттерны RBAC:

- **Использование default service account** без ограничений и с включенным automountServiceAccountToken: true.
- **Выдача ClusterRole с правами cluster-admin** прикладным приложениям.
- **Использование "звездочек" (wildcards):** verbs: ["*"], resources: ["*"]. Это выдает полные права на все ресурсы.

- **Права на создание Pods/Deployments/DaemonSets** в системных неймспейсах (kube-system).
- **Права на чтение Secrets** (resources: ["secrets"], verbs: ["get", "list", "watch"]).

Сценарий атаки

- Атакующий внутри контейнера находит токен.
- Используя curl или скачанный kubectl, проверяет права (kubectl auth can-i --list).
- Обнаруживает право create pods.
- Создает YAML-манифест привилегированного пода, который монтирует / хоста.
- Запускает этот под и получает root-доступ к узлу.

Рекомендации

- Отключать automountServiceAccountToken: false везде, где токен не нужен приложению.
- Жестко минимизировать выдачу «мощных» прав.
- Использовать принцип наименьших привилегий.
- Изолировать критичные нагрузки: запускать поды с высокими правами на выделенных узлах (Taints/Tolerations) и не смешивать их с менее доверенными пользовательскими приложениями.

Для целей защиты важно связать это с темой побега: даже если контейнер «заперт» на уровне Linux, но через токен он способен создавать свои привилегированные поды или менять спецификации других подов, это все равно квалифицируется как **практический escape-вектор**.

5. Эшелонированная защита: Admission Controllers и Pod Security

Контроль плоскости управления критичнее поиска «скрытых настроек» вручную. Задача архитектора безопасности — не просто ловить флаги в YAML, а внедрить политики, которые физически не позволят применить опасную конфигурацию.

Pod Security Admission (PSA)

Начиная с Kubernetes 1.25+, PSA стал стандартом (GA). Он реализует три уровня политик Pod Security Standards:

- **Privileged**: Полная свобода (небезопасно).
- **Baseline**: Базовые ограничения, запрещающие явные уязвимости, но допускающие стандартные настройки.
- **Restricted**: Строгие ограничения (запет privileged, hostNetwork, hostPath, требование runAsNonRoot, сброс capabilities).

NSA/CISA и CNCF рекомендуют использовать Admission-контроллеры (PSA или внешние вебхуки, такие как OPA Gatekeeper или Kyverno) для того, чтобы:

- Запретить запуск привилегированных Pods.
- Блокировать опасные hostPath и host* опции.
- Внедрять проверку образов (Image Scanning) и их подписей на этапе адмиссии.

Сдвиг парадигмы: Defense-in-Depth

Отсутствие или слабая конфигурация слоя Admission обычно означает, что любая ошибка разработчика, проблема в CI/CD или злоупотребление правами приведет к появлению «дырявых» подов.

Практики 2022–2026 годов сходятся во мнении: основная задача — защитить Control Plane и его политики.

- **Защита Kubernetes API:** Аутентификация, mTLS, отключение анонимного доступа, сегментация сети.
- **Разделение Tenancy:** Жесткое разделение по Namespace и ролям.
- **Принудительное применение политик:** Baseline/Restricted профили.

Подход NSA/CISA формулируется как **Defense-in-Depth**: мы исходим из предположения, что отдельный Pod *периодически будет скомпрометирован*. Наша задача — минимизировать радиус поражения (blast radius), а не пытаться сделать «непробиваемый» контейнер. Compromise одного Pod'a не должен означать compromise всего узла или кластера.

6. Методология Red Team: Проверка Escape-Risk

Как специалисты по наступательной безопасности проверяют стойкость кластера? Мы не ищем 0-day эксплойты сразу. Мы идем по чек-листу конфигураций.

Шаг 1. Enumeration (Разведка) внутри контейнера

Первый шаг — понять, насколько текущий контейнер близок к хосту.

- **Анализ пространств имен:** Проверка /proc/ 1/ cgroup, /proc/ self/ status и /proc/ net/ для выявления hostPID или hostNetwork.
- **Проверка Security Context:**
 - ID пользователя (id): root (uid=0) или обычный пользователь?
 - Capabilities (capsh --print или анализ /proc/ self/ status): ищем CAP_SYS_ADMIN, CAP_NET_ADMIN.
 - LSM: работает ли AppArmor/SELinux?

- **Инвентаризация маунтов:**

- Команда mount или cat /proc/ self/ mounts.
- Ищем: /var/ run, /run/ containerd, /var/ lib/ kubelet, /etc/ kubernetes, /dev/ хоста.

- **Поиск сокетов:**

- find / -name *.sock.
- Цель: docker[.]sock, containerd[.]sock, cri-socket, сокет kubelet, CSI/CNI.

Цель этапа: Собрать карту всех конфигурационных «рычагов», которые могут дать доступ к node или к Kubernetes API.

Шаг 2. Проверка доступа к Kubernetes API и метаданным

Далее выясняем, что можно сделать с токеном сервис-аккаунта.

- **Поиск кредитов:** Чтение /var/ run/ secrets/ kubernetes.io/ serviceaccount (token, ca.crt, namespace).

- **Запросы к API:**

- Может ли Pod листить (list), создавать (create) или обновлять (update/patch) ресурсы?
- Интересуют: Pods, Deployments, Secrets, Roles, RoleBindings, Nodes, DaemonSets.

- **Cloud Metadata:**

- Попытка обратиться к http:// 169.254.169.254 (AWS/Azure/GCP).
- Цель: получить временные IAM-креды ноды и выйти за пределы кластера в облачную консоль.

Часто именно через чрезмерные права сервис-аккаунта (например, возможность изменить RoleBinding) Red Team демонстрирует путь от компрометации одного приложения к полному захвату кластера (Cluster Takeover).

Шаг 3. Анализ контроля Admission и Pod Security

Проверяем, какие политики реально enforced (применяются принудительно).

- Есть ли Pod Security Admission в неймспейсе? На каком уровне (privileged, baseline, restricted)?
- Есть ли внешние вебхуки (OPA, Kyverno)? Можно ли их обойти?
- **Сценарий проверки:** Может ли злоумышленник (через RCE или украденный credential) создать Pod со следующими свойствами:
 - privileged: true
 - hostPath на /
 - hostPID/hostNetwork
 - Опасные capabilities.

Если это возможно в production-namespace, Red Team помечает это как **критический риск container escape** даже без использования CVE.

Шаг 4. Проверка Runtime и Node-уровня

С точки зрения узла оцениваем:

- Есть ли на ноде другие рабочие нагрузки с --privileged или широкими hostPath.
- Как настроены seccomp/AppArmor/SELinux: дефолтные политики, кастомные или всё работает в режиме «unconfined» (без ограничений).

Публичные примеры (отчеты CyberArk, серии пентестов Kubernetes) демонстрируют: комбинация «ошибка в deploy + слабый admission + привилегированный Pod» дает возможность выйти на root node и закрепиться в системе.

7. Практический чек-лист Hardening (Без эксплойтов)

Этот чек-лист необходимо использовать для проверки кластеров перед выводом в продакшн.

Уровень Node и Ядра

Актуальное ядро Linux и контейнерный runtime (Docker/Containerd/CRI-O) с настроенным процессом регулярного патчинга.

Включенные LSM: AppArmor или SELinux в режиме enforcing.

Seccomp: Включены профили seccomp по умолчанию для всех контейнеров.

Ограничение устройств: Минимальный набор устройств в /dev/, отсутствие лишних device-маунтов.

Уровень Контейнерного Рантайма и образов

Запрет --privileged: Полный отказ от привилегированных контейнеров.

Capabilities: Принцип «drop ALL, add по необходимости» (белый список).

Минимизация hostPath: Использовать только readOnly-маунты и только конкретных поддиректорий. Запрет /, /etc/, /root/, /var/ lib/ kubelet.

Запрет маунта сокетов: docker[.]sock и containerd[.]sock запрещены для рабочих приложений. Для служебных задач (сборка, мониторинг) использовать выделенные изолированные решения (Kaniko, unprivileged builds).

Immutability: Использование readOnlyRootFilesystem: true там, где это возможно.

Образы: Minimal base images (distroless/alpine), отсутствие шелла и лишних утилит.

Уровень Kubernetes: Pod Security, RBAC, Admission

Pod Security Admission: Включен. По умолчанию уровень restricted во всех production-namespaces.

RBAC: Принцип наименьших привилегий. cluster-admin только для платформенной команды.

Service Accounts: automountServiceAccountToken: false по умолчанию. Монтирование только там, где реально нужен доступ к K8s API.

Admission Webhooks: Настроены правила для запрета привилегированных подов, hostPath, hostNetwork/hostPID, enforcement использования runAsNonRoot.

Уровень Сети и Секретов

Сегментация сети: Network Policies с явным deny-by-default (ingress/egress) между неймспейсами.

Control Plane: Доступ к Kubernetes API и etcd ограничен, не выставлен в интернет (или защищен через VPN/Bastion).

Секреты: Использование Kubernetes Secrets (с шифрованием etcd at rest) или внешних KMS (Vault). Запрет передачи секретов через Environment Variables.

Мониторинг и Детекция

Audit Logs: Включен аудит Kubernetes API. Отслеживание создания/обновления Privileged Pods, ролей, биндингов.

Runtime Security: Использование инструментов типа Falco/Tetragon (eBPF) для логирования системных вызовов внутри контейнеров (доступ к sensitive файлам, запуск шелла, сетевые соединения).

8. Формат отчета Red Team по контейнерной среде

Для эффективного взаимодействия между Red Team и командами DevOps/SRE/SecOps, отчет должен сразу подсвечивать конфигурационные риски. Идеальная структура отчета:

1. Executive Summary

Краткое описание успеха операции: удалось ли выйти на уровень Node/Cluster? Через какие высокоуровневые векторы (например, "Privileged Pod в namespace dev

позволил захватить узел, а затем через токен SA получить полный контроль над кластером").

2. Threat Model и Scope

Описание проверенных неймспейсов, кластеров и окружений. Какие роли и сервис-аккаунты попали в скоуп.

3. Технические находки (по категориям)

- **Runtime-misconfig:** Привилегированные поды, опасные hostPath, режимы host*, отсутствие seccomp/AppArmor.
- **Kubernetes-misconfig:** RBAC (чрезмерные права SA, cluster-admin), отсутствие Pod Security Admission, слабые Admission Webhooks.
- **Supply Chain:** Устаревшие образы, вшитые секреты, отсутствие сканирования при деплое.

4. Цепочки эскалации (Attack Paths)

Детальное описание путей атаки.

Пример: «RCE в Pod X → использование docker[.]sock → запуск privileged контейнера с маунтом корня → root на node Y → использование кредов из /etc/kubernetes/ → Cluster Admin». Важно описывать шаги концептуально, без публикации готовых эксплойтов, но с достаточной детализацией для воспроизведения инженерами.

5. Оценка риска

Severity (критичность) и Likelihood (вероятность) с учетом контекста (Production vs Dev, наличие сетевой связности, наличие мониторинга).

6. Рекомендации (Remediation)

- **DevOps/SRE:** Конкретные правки манифестов (изменение securityContext, удаление hostPath, добавление лейблов для PSA).
- **Infrastructure:** Изменения в RBAC (разделение ролей), обновление версий K8s/Runtime.
- **Security:** Включение политик Admission, настройка Falco/audit-логов.

7. Detection Opportunities

На что должны были сработать алерты: запуск шелла в контейнере, создание привилегированного пода, чтение токена SA, обращение к метаданным облака.

Вывод

Безопасность контейнеров в 2026 году — это дисциплина конфигурации, а не борьбы с эксплойтами.

- **Контейнер разделяет ядро хоста**, поэтому изоляция слабее по определению (в отличие от VM).
- **Большинство инцидентов** — это следствие избыточных привилегий, опасных маунтов и прав в Kubernetes, а не 0-day уязвимостей.
- **Контроль плоскости управления** (RBAC, Admission, Secrets) критичнее попыток отловить каждый "неправильный" флаг вручную.

Выход из контейнера — это прежде всего следствие архитектурных решений. Работа Red Team — это системная проверка этих решений на прочность. Используйте приведенные чек-листы и модели угроз, чтобы закрыть векторы атак до того, как ими воспользуются злоумышленники.

FAQ

В: Спасает ли использование Distroless-образов от побега из контейнера?

О: Частично. Distroless-образа (без шелла и пакетного менеджера) усложняют жизнь атакующему на этапе "житья внутри" (Living off the Land), так как нет bash, curl, apt. Но если есть возможность выполнить бинарный код или скрипт (через интерпретатор приложения), и контейнер сконфигурирован небезопасно (например, --privileged), побег всё равно возможен. Это мера Defense-in-Depth, но не серебряная пуля.

В: Нужно ли запрещать automountServiceAccountToken для всех подов?

О: Да, это отличная практика. По умолчанию токен монтируется всем. Рекомендуется ставить false глобально или в дефолтном SA, и включать true только для тех подов (например, Ingress Controller, Operator), которым действительно нужно общаться с K8s API.

В: Является ли hostPath уязвимостью, если он read-only?

О: Зависит от пути. Read-only маунт /etc/ всё равно позволяет атакующему прочитать /etc/ shadow (если user — root) или конфигурацию системы, что помогает в дальнейшей атаке. Read-only маунт логов или конфигураций приложения менее опасен, но лучше использовать ConfigMap, Secret или EmptyDir.

В: Почему Red Team предпочитает ошибки конфигурации, а не Kernel Exploit?

О: Эксплойты ядра (Kernel Exploits) часто нестабильны, могут вызвать падение узла (Kernel Panic), зависят от конкретной версии ядра и архитектуры. Ошибки конфигурации (например, доступный docker[.]sock) работают стабильно, предсказуемо и не "шумят" так сильно на уровне ядра.

В: Достаточно ли Pod Security Admission уровня baseline?

О: Уровень baseline защищает от самых явных проблем (privileged, host namespaces), но пропускает некоторые capabilities и не требует runAsNonRoot. Для критичных систем и production-окружений настоятельно рекомендуется уровень restricted.

В: Как защитить docker[.]sock, если он нужен для CI/CD (Docker-in-Docker)?

О: Избегайте проброса сокета (-v /var/ run/ docker[.]sock:/var/ run/ docker[.]sock). Вместо этого используйте альтернативы: Kaniko (сборка без демона), Podman (rootless контейнеры) или специальные решения для изолированных билдеров. Если доступ к сокету неизбежен, изолируйте такие поды на отдельных узлах через Taints/Tolerations и строго ограничьте их сетевой доступ.